

Instant Size Count — what's live, and the full process to finish the camera

Last updated 2026-07-21. Companion to [docs/superpowers/specs/2026-07-21-instant-size-count-phase0.md](#).

What is LIVE right now (shipped 2026-07-21)

A working **manual per-vendor size counter**, review-only (it does **not** change inventory):

- **Server:** `size_count_sessions` + immutable event ledger + adjustment audit (mig 0283, on prod, pgTAP-verified). Bearer API: `POST /api/v1/size-counts` (start), `POST /api/v1/size-counts/[id]/events` (idempotent append), `POST /[id]/complete` (lock), `GET /[id]` (session + per-size tally). All behind the `instant_size_count` module.
- **Mobile (OTA'd):** a "Size count" drawer item → start a count (optional vendor/reference) → live tap-to-count grid of the 9 sizes (tap +1, long-press -1, Undo) → Finish (locks the list). Counts queue through the offline outbox and sync idempotently.
- **Enabled for:** Demo Co (`71b27a4a-7948-4638-bc3f-535974713bd2`).

To use it now

1. On your phone, **force-close and reopen** the app to pull the OTA.
2. Switch workspace to **Demo Co** (or enable the module for your production org — see below).
3. Open the drawer → **Size count** → **Start counting** → tap sizes as you go → **Finish**.

To enable it for your production org

Settings → Modules → Premium → toggle **Instant size count** on (as an admin of that org). That's the only step; the drawer item and API turn on immediately.

The camera auto-detector — the full process to finish

The manual counter above **is the shell** the camera drops into. The camera replaces "tap a size" with "the model saw an L sticker cross the gate → +1 L" — same session, same event ledger, same review list. Getting there is a real ML + native-build effort. Here is the honest, complete sequence, and who owns each step.

Why it can't just be an OTA (the constraint, restated)

- The camera needs a **native module** (react-native-vision-camera + an on-device TFLite runtime). Native code ships only in an **EAS build** → **TestFlight/store**, never OTA (`runtimeVersion=appVersion`).
- After that native shell is installed, the **trained model file is an asset** — so model updates and tuning DO ship via OTA on top of the installed build.

Step 1 — Capture a dataset · **OWNER (I scaffold the tool)**

The model must be trained on **real** size stickers in real conditions. One reference photo is not a dataset.

- I build an opt-in **capture mode** in the app that records frames + you tag the size (this becomes labeled training data, stored with consent + retention).
- You (warehouse staff) capture **hundreds to low-thousands** of examples across: every size, sticker brands/fonts, wear/damage, under plastic, glare, low/bright light, motion blur, angles + rotation, light/dark/patterned garments, multiple stickers per frame, and **hard negatives** (buttons, logos, neck labels — things that are *not* a size sticker).
- Split by session/box/day/device so the model doesn't "cheat."

Step 2 — Label + validate · **OWNER (I provide the schema + tooling)**

Draw a box around each sticker + its size class; mark visibility/blur/glare/occlusion. I set up the annotation schema, dataset versioning, and a locked test set that never trains.

Step 3 — Train + export · **OWNER's ML step (I scaffold the pipeline + configs)**

- Train the recommended **two-stage** model (detect the circular sticker → classify the 9 sizes) with an explicit **"not sure" abstain** so it never guesses M-vs-W or XL-vs-XXL.
- Quantize + export to **.tflite** (Android) and **Core ML** (iOS).
- Measure precision/recall **per size** against the locked test set. I can't do this — there's no data and no model here — but I provide the training scripts, the confusion-matrix harness, and the release gates.

Step 4 — Add the native camera + build to TestFlight · **ME (code) → OWNER (build)**

- I add **react-native-vision-camera** + **react-native-fast-tflite** + **react-native-worklets-core** + the VisionCamera config plugin to **app.config.ts**, and a **frame-processor** that runs the model per frame and emits **{size, confidence, box}** — wired into the *same* counting screen (Rapid Pass gate + Box Overview).
- This is a **version bump + new EAS build**: you run **pnpm release:ios** → it auto-submits to TestFlight → you install it.
- First TestFlight build can ship with a **stub/placeholder model** so you can validate the camera plumbing before the real model exists.

Step 5 — Drop in the model + tune · **ME (OTA) with your on-device feedback**

Once you have a trained **.tflite**/Core ML: the model file + any JS tuning (gate position, confidence threshold, dedupe window) ship via **pnpm release:ota** on top of the TestFlight build. Iterate from what you see on the device.

Step 6 — Validate on-device · **OWNER (only real hardware can)**

Measure the release gates on your phone: per-size accuracy, duplicate-count rate, missed-count rate, FPS/thermal. **Rapid Pass** (move each garment past a count gate) is the accurate primary mode; **Box Overview** stays a warned beta until its duplicate rate is proven. I can't produce these numbers — they only exist on the device.

Step 7 — Roll out

Feature-flag per org, pilot in one warehouse, then widen. The future **Smart Label** (a QR/DataMatrix/NFC tag next to the printed size) is the only way to get *absolute* per-garment identity — an optional later ops project, not required for the counter.

Division of labor, at a glance

Step	Me (in-repo)	You (hardware/data)
Manual counter	✅ shipped	use it now
Capture tool	build it	run it in the warehouse
Annotation schema + pipeline	scaffold it	label + train
Native camera + frame-processor	code it	trigger the EAS build
Model integration	OTA the asset	give device feedback
On-device benchmarks	provide the harness	run on real phones

The one thing that unblocks everything else is **Step 1 — the dataset**. Say the word and I'll build the in-app capture tool next so you can start collecting sticker footage while the manual counter is already earning its keep.